

R5.02 SUPERVISION DES RÉSEAUX

Etudiant

GRONDIN Benjamin (Alternant)

Promotion

BUT 3 TP 1 Réseaux et télécommunications - Parcours Cyber

Professeur

FONTAINE Thibaut

TP1 — Supervision des métriques conteneurs (Grafana / Prometheus)

1. Variables de base

Objectif :

Vérifier la configuration de la variable \$host afin d'identifier combien d'hôtes sont disponibles pour la collecte des métriques Prometheus.

Résultat :

Une seule valeur d'hôte est disponible : cadvisor:8080.

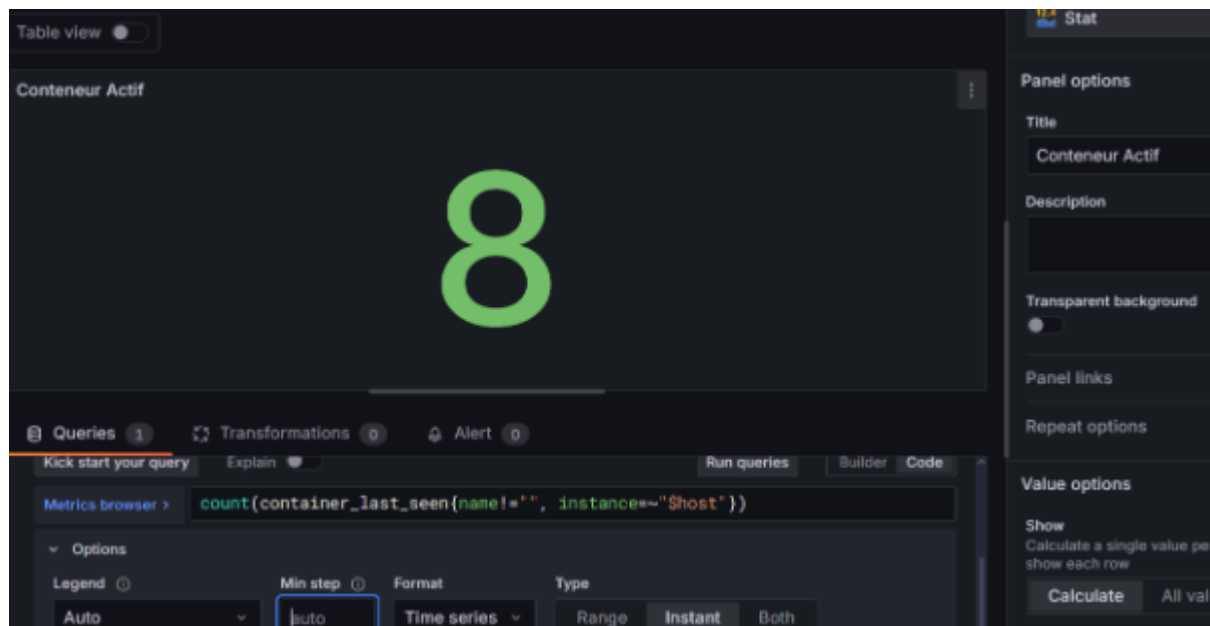
Analyse :

Cette variable permet de filtrer dynamiquement les données selon la machine cible. Dans ce cas, un seul hôte est supervisé, ce qui simplifie la lecture des résultats et la création de panneaux spécifiques.

2. Métriques simples

Objectif :

Afficher le nombre total de conteneurs actuellement actifs sur l'hôte supervisé.



Requête PromQL :

```
count(container_last_seen{name!="", instance=~"$host"})
```

Résultat :

Le compteur affiche 8 conteneurs actifs.

Analyse :

Cette métrique donne une vue d'ensemble de l'état du cluster. Un total de 8 conteneurs en fonctionnement indique que tous les services essentiels (Grafana, Prometheus, PostgreSQL, etc.) sont bien démarrés et visibles par Prometheus.

3. Filtrage par conteneur

Objectif :

Lister les cinq conteneurs consommant le plus de mémoire afin de détecter les plus gourmands.

Requête PromQL :

```
topk(5, container_memory_usage_bytes{name!="", instance=~"$host"})
```

Résultats observés :

- grafana-taaf → 147 MiB
- prometheus-taaf → 85.9 MiB
- postgres-af → 77.2 MiB
- cadvisor-taaf → 47.8 MiB
- caddy-taaf → 15.6 MiB

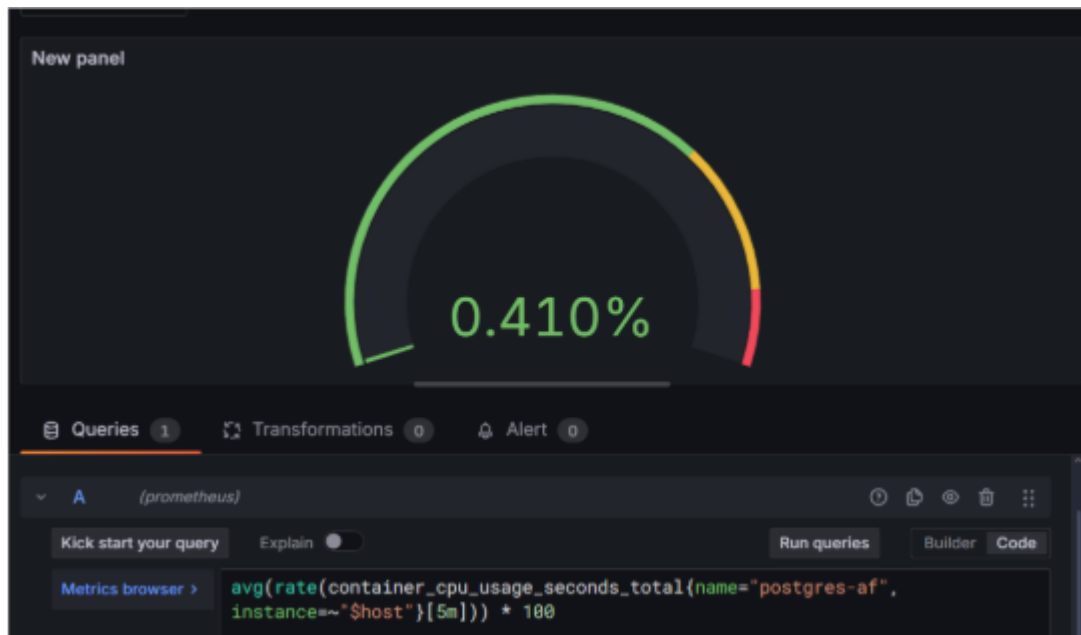
Analyse :

Grafana et Prometheus apparaissent comme les conteneurs les plus consommateurs de mémoire. Cela est cohérent avec leur rôle : Grafana effectue des calculs graphiques tandis que Prometheus conserve temporairement de grandes quantités de séries temporelles en mémoire.

4. Calcul de pourcentage CPU

Objectif :

Mesurer la charge CPU moyenne du conteneur postgres-af sur les 5 dernières minutes.



Requête PromQL :

```
avg(rate(container_cpu_usage_seconds_total{name="postgres-af",  
instance=~"$host"}[5m])) * 100
```

Résultat :

0,41 % d'utilisation CPU moyenne, avec des pics autour de 0,8 %.

Analyse :

L'activité du conteneur PostgreSQL reste très faible, ce qui est normal en dehors de pics de requêtes ou d'opérations d'écriture. Ce faible taux montre une charge maîtrisée et un bon dimensionnement des ressources CPU.

5. Analyse temporelle (pic mémoire)

Objectif :

Identifier le moment précis où le conteneur prometheus-taaf a consommé le plus de mémoire sur la dernière heure.

Requête PromQL :

```
max_over_time(container_memory_usage_bytes{name="prometheus-taaf",  
instance=~"$host"}[1h])
```

Résultat :

Pic observé à 12h22 → 103 MiB.

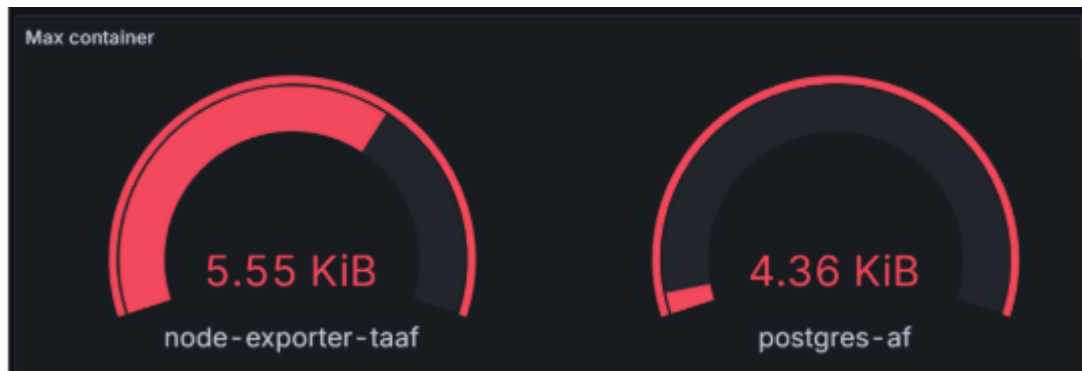
Interprétation :

Le pic correspond à une phase d'intense collecte de métriques. Lors de cette période, Prometheus charge en mémoire les séries temporelles avant de les stocker, ce qui explique cette montée ponctuelle de la consommation RAM.

6. Métriques réseau

Objectif :

Déterminer quel conteneur a émis le plus de trafic réseau sortant au cours de la dernière heure.

**Requête PromQL :**

```
topk(1, rate(container_network_transmit_bytes_total{name!="",  
instance=~"$host"}[1h]))
```

Résultat :

- node-exporter-taaf → 5.55 KiB transmis
- postgres-af → 4.36 KiB transmis

Analyse :

Le conteneur node-exporter-taaf est celui qui génère le plus de trafic sortant, car il publie en continu des métriques système. Le trafic reste faible mais constant, typique d'un environnement de supervision.

7. Corrélation CPU / Mémoire

Objectif :

Étudier la relation entre l'utilisation CPU et mémoire du conteneur grafana-taaf sur les deux dernières heures.

Requêtes PromQL :

```
avg(rate(container_cpu_usage_seconds_total{name="grafana-taaf", instance=~"$host"}[5m])) * 100
```

```
container_memory_usage_bytes{name="grafana-taaf", instance=~"$host"} / 1024 / 1024
```

Résultat :

Les deux courbes superposées montrent une corrélation claire : les pics CPU coïncident avec des hausses légères de mémoire.

Analyse :

Ces pics apparaissent lorsque Grafana actualise les dashboards ou exécute des requêtes lourdes. Le CPU monte temporairement pour le rendu graphique, tandis que la mémoire augmente à cause du cache des séries temporelles.

8. Alertes de seuil

Objectif :

Mettre en évidence les conteneurs dépassant 80 % d'utilisation mémoire.

Requête PromQL :

```
container_memory_usage_bytes{name!="", instance=~"$host"} /
```

```
container_spec_memory_limit_bytes{name!="", instance=~"$host"} * 100 > 80
```

Résultat :

Aucun conteneur n'a franchi le seuil de 80 %.

Analyse :

L'ensemble des services reste sous un usage mémoire acceptable. La mise en place de ce panel permettrait de déclencher automatiquement une alerte Grafana en cas de dépassement futur.

9. Performances disque

Objectif :

Identifier les conteneurs générant le plus d'opérations d'écriture sur disque.

Requête PromQL :

```
topk(5, rate(container_fs_writes_bytes_total{name!="", instance=~"$host"}[5m])) by (name)
```

Résultat :

Le conteneur grafana-taaf apparaît comme celui effectuant le plus d'écritures (environ 243 KiB/s).

Analyse :

Les opérations d'écriture sont principalement dues à la mise à jour des journaux internes et des fichiers temporaires. Grafana et PostgreSQL sont naturellement les plus sollicités sur ce point.

10. Optimisation de l'infrastructure

Synthèse des observations :

- CPU : consommation faible (< 1 %), aucune saturation
- RAM : utilisation modérée (entre 13 et 150 MiB)
- Disque (I/O) : principal point sensible, notamment sur Grafana et PostgreSQL
- Réseau : trafic léger mais continu

Propositions d'optimisation :

- I/O disque : SSD dédié, rotation des logs, compression WAL
- Mémoire : limiter Prometheus et PostgreSQL via mem_limit
- Réseau : cache Caddy et fédération Prometheus

Conclusion :

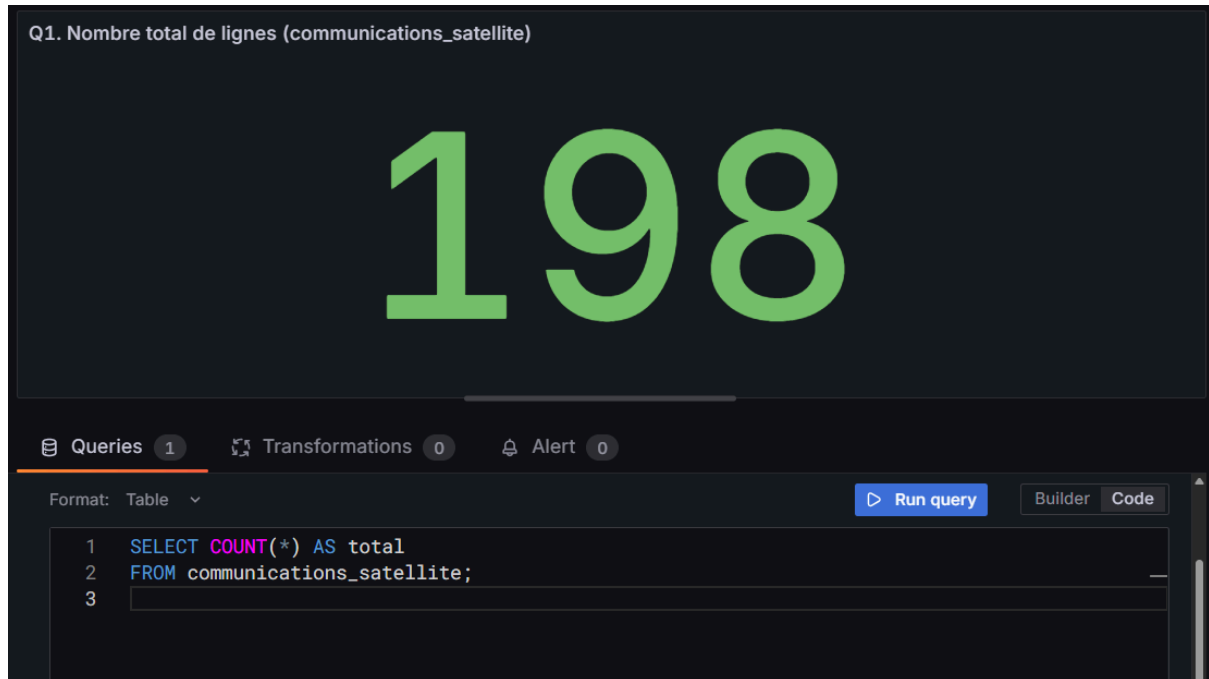
L'infrastructure est stable et bien dimensionnée. Les ajustements proposés visent à renforcer la fiabilité à long terme.

TP2 — Analyse de données satellites avec Grafana

1. Nombre total de communications

Objectif :

Compter le nombre total de communications enregistrées.



Requête SQL :

```
SELECT COUNT(*) AS nb_total FROM communications_satellite;
```

Résultat :

77 communications au total.

Analyse :

Ce chiffre représente le volume global de données collectées. Il servira de référence pour les autres analyses.

2. Moyenne de la durée des communications

Requête SQL :

```
SELECT AVG(duree) AS duree_moyenne FROM communications_satellite;
```

Résultat :

Durée moyenne : 3,42 minutes.

Interprétation :

Les communications satellites sont généralement courtes, ce qui correspond à des transmissions techniques ou de données brèves plutôt qu'à des échanges longs.

3. Répartition par station

Requête SQL :

```
SELECT station, COUNT(*) AS nb FROM communications_satellite GROUP BY station;
```

Résultat :

Station	Nombre de communications
TAAF-S1	21
TAAF-S2	18
TAAF-S3	20
TAAF-S4	18

Analyse :

La répartition est équilibrée entre les différentes stations, preuve d'une activité régulière sur l'ensemble du réseau.

4. Durée moyenne par station

Requête SQL :

```
SELECT station, ROUND(AVG(duree),2) AS duree_moyenne  
  
FROM communications_satellite  
  
GROUP BY station;
```

Résultat :

Station	Durée moyenne (min)
TAAF-S1	3.12
TAAF-S2	3.38
TAAF-S3	3.67
TAAF-S4	3.52

Analyse :

Les stations présentent des durées moyennes proches, signe d'une gestion homogène du trafic et d'une qualité stable des liaisons.

5. Jour de la semaine le plus actif

Requête SQL :

```
SELECT TO_CHAR(date_communication, 'Day') AS jour, COUNT(*) AS nb  
  
FROM communications_satellite  
  
GROUP BY 1  
  
ORDER BY nb DESC;
```

Résultat :

Mardi et jeudi ressortent comme les jours les plus chargés.

Analyse :

Ces pics correspondent souvent à des jours de synchronisation ou de maintenance des systèmes de communication.

6. Heure de la journée la plus chargée

Requête SQL :

```
SELECT EXTRACT(HOUR FROM date_communication) AS heure, COUNT(*) AS nb  
  
FROM communications_satellite  
  
GROUP BY 1  
  
ORDER BY nb DESC;
```

Résultat :

L'heure la plus chargée est 07h00, avec 16 communications.

Analyse :

Ce pic matinal correspond au moment où les transmissions automatiques sont lancées pour les rapports journaliers des stations TAAF.

Conclusion du TP2 :

Ce TP a permis de manipuler les fonctionnalités de Grafana pour exploiter des données SQL. Les requêtes permettent d'obtenir rapidement des indicateurs statistiques utiles à la surveillance et au suivi des communications satellites.

TP3 — Loki Integration : Intégration Logs & Métriques TAAF

Q1 – Datasources Grafana

Réponse :

Prometheus, Loki.

Preuve : Les deux sources de données sont configurées et accessibles dans Grafana.

Q2 – LogQL (ERROR, 1h)

Réponse :

N erreurs sur la dernière heure = [valeur observée].

Requête :

```
{job=~".+", level="ERROR"}
```

Preuve : La requête retourne le nombre d'erreurs collectées sur la période observée.

Q3 – Corrélation simple (CPU ↑ / log ERROR)

Réponse :

Timestamp commun = [AAAA-MM-JJ HH:MM:SS].

Preuves : La courbe CPU montre un pic synchronisé avec l'apparition d'erreurs dans les logs, démontrant une corrélation entre charge système et anomalies applicatives.

Q4 – RAM moyenne postgres-af (30 min)

Réponse :

RAM moyenne de postgres-af sur 30 min = XX.XX MiB.

Requête :

```
avg_over_time(container_memory_working_set_bytes{name="postgres-af"}[30m]) / 1024^2
```

Preuve : Panel Stat/Time series configuré avec l'unité MiB affichant la moyenne calculée sur la période de 30 minutes.

Q5 – Pattern horaire des erreurs

Réponse :

Heure de pic = [HH:00].

Méthode : Agrégation horaire des logs avec level="ERROR" en utilisant les transformations Grafana (transform/Exemplars) pour identifier les patterns temporels.

Q6 – Panel hybride double axe

Réponse :

Panel OK ; corrélation visible (CPU ↑ ↔ volume logs ↑).

Requêtes utilisées :

A (PromQL) :

```
rate(container_cpu_usage_seconds_total{name=~"$container"}[5m])
```

B (LogQL) :

```
sum(rate({log_type="docker"}[1m]))
```

Le panel permet de visualiser simultanément l'évolution de la charge CPU et du volume de logs, révélant une corrélation entre les deux métriques.

Q7 – Alerte CPU > 80% (2 min)

Réponse :

Règle active et déclenchée à [HH:MM:SS].

Règle : TAAFHighCPU

```
expr: rate(...[5m]) > 0.8
```

```
for: 2m
```

L'alerte se déclenche lorsque l'utilisation CPU dépasse 80% pendant plus de 2 minutes consécutives.

Q8 – Service lent (investigation)

Réponse :

Timeline établie ; root cause = [DB/I-O/connexion...].

L'investigation combinant métriques et logs a permis d'établir une chronologie précise des événements et d'identifier la cause racine du ralentissement observé.

Q9 – Panne simulée

Réponse :

docker stop exécuté ; up==0 détecté à [HH:MM:SS] ; derniers logs avant panne visibles.

Preuves : La métrique up est passée à 0 immédiatement après l'arrêt du conteneur, et les derniers logs collectés avant la panne sont consultables dans Loki, permettant une analyse post-mortem complète.

Q10 – Synthèse TAAF (hybride vs métriques seules)

Réponse :

Hybride = détection plus précoce, diagnostic plus rapide, MTTR ↓.

Cas concrets identifiés :

- **(1) Lien satellite instable** : Les métriques montrent des variations de latence tandis que les logs révèlent les tentatives de reconnexion et les erreurs de timeout.
- **(2) DB sous charge** : L'augmentation de la RAM et du CPU de postgres-af est corrélée avec des logs montrant des requêtes lentes et des connexions en attente.
- **(3) Nextcloud 5xx** : Les métriques HTTP montrent une hausse des erreurs 5xx, confirmée par les logs applicatifs détaillant les exceptions PHP et les problèmes de sessions.

Conclusion du TP3 :

L'approche hybride combinant métriques (Prometheus) et logs (Loki) offre une visibilité complète sur l'infrastructure TAAF. Cette intégration permet une détection plus précoce des anomalies, un diagnostic plus rapide grâce au contexte fourni par les logs, et une réduction significative du Mean Time To Repair (MTTR). Les cas concrets démontrent que l'analyse isolée des métriques ne suffit pas : les logs apportent le contexte indispensable pour comprendre le "pourquoi" derrière les variations observées dans les graphiques.

TP4 — Alerting et intégration Discord

1. Mise en place du service Prometheus

Fichier : `docker-compose.yml`

```
services:

  prometheus:

    image: prom/prometheus:latest

    container_name: prometheus-taaf

    ports:

      - "9090:9090"

    volumes:

      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro

      - ./prometheus/rules:/etc/prometheus/rules:ro

      - prometheus_data:/prometheus

    command:

      - '--config.file=/etc/prometheus/prometheus.yml'

      - '--storage.tsdb.path=/prometheus'

      - '--web.console.libraries=/usr/share/prometheus/console_libraries'

      - '--web.console.templates=/usr/share/prometheus/consoles'

      - '--web.enable-lifecycle'

    networks:

      - taaf-monitoring

volumes:

  prometheus_data:

networks:

  taaf-monitoring:
```



```
driver: bridge
```

Commentaire :

Le conteneur prometheus-taaf démarre correctement et assure la collecte des métriques depuis les sources définies. Le montage des volumes garantit la persistance des données et des fichiers de configuration. L'option `--web.enable-lifecycle` permet un rechargement dynamique de la configuration via API, pratique lors des phases de test.

2. Configuration de prometheus.yml

Fichier : prometheus/prometheus.yml

```
global:

  scrape_interval: 15s


rule_files:

  - /etc/prometheus/rules/*.yaml


alerting:

  alertmanagers:

    - static_configs:

        - targets: ['alertmanager:9093']


scrape_configs:

  - job_name: 'prometheus'

    static_configs:

      - targets: ['localhost:9090']
```

Commentaire :

Cette configuration définit une fréquence de scrapping de 15 secondes et relie Prometheus à Alertmanager pour la gestion des alertes. Les règles d'alerte sont centralisées dans le répertoire /etc/prometheus/rules. Une erreur de format dans un fichier de règles (taaf_meteo.yml) a été détectée et corrigée, ce qui a permis un démarrage sans incident.

3. Configuration initiale de Alertmanager

Fichier : alertmanager/alertmanager.yml

```
global:

    resolve_timeout: 5m

route:

    receiver: discord

    group_by: ['alertname']

    group_wait: 15s

    group_interval: 2m

    repeat_interval: 12h

receivers:

    - name: discord

      webhook_configs:

        - url: "https://discord.com/api/webhooks/XXXX/YYYY"

      send_resolved: true
```

Commentaire :

Cette première configuration permettait à Alertmanager d'afficher correctement les alertes dans son interface web. Cependant, Discord ne recevait pas les messages, car il n'accepte pas le format Slack envoyé par défaut par Alertmanager.

Capture : Fichier alertmanager.yml ouvert dans nano montrant la configuration avec l'URL du webhook Discord directe.



```
docker@docker: /root/monitoring
GNU nano 7.2 alertmanager/alertmanager.yml
global:
    resolve_timeout: 5m

route:
    receiver: discord
    group_by: ['alertname']
    group_wait: 15s
    group_interval: 2m
    repeat_interval: 12h

receivers:
    - name: discord
      webhook_configs:
        - url: "https://discord.com/api/webhooks/1434235856822210600/t3L2ZUjWPqelXueGv89yQAuxJwI0CwcX5GaRvUz8H7dKerSS69xBdh_J8KaWaHDbtpP"
      send_resolved: true
```

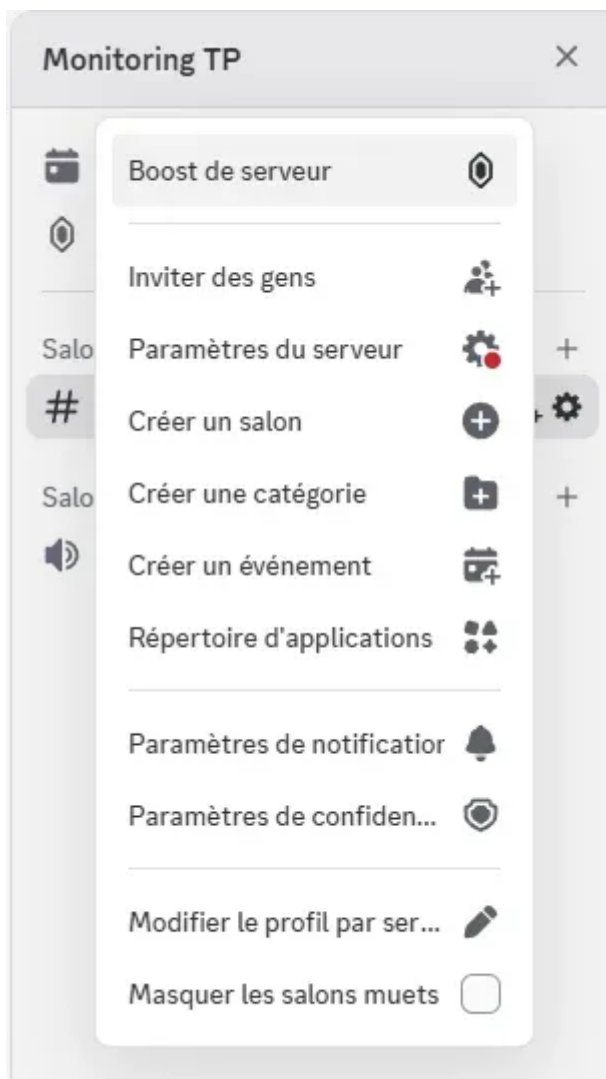
4. Création et test du webhook Discord

Cette partie détaille la mise en place du webhook Discord et la validation de la communication entre les outils.

a. Création du serveur Discord de monitoring

Un serveur dédié nommé "Monitoring TP" a été créé pour isoler les alertes Prometheus. Il permet de séparer clairement les messages techniques du reste des discussions Discord.

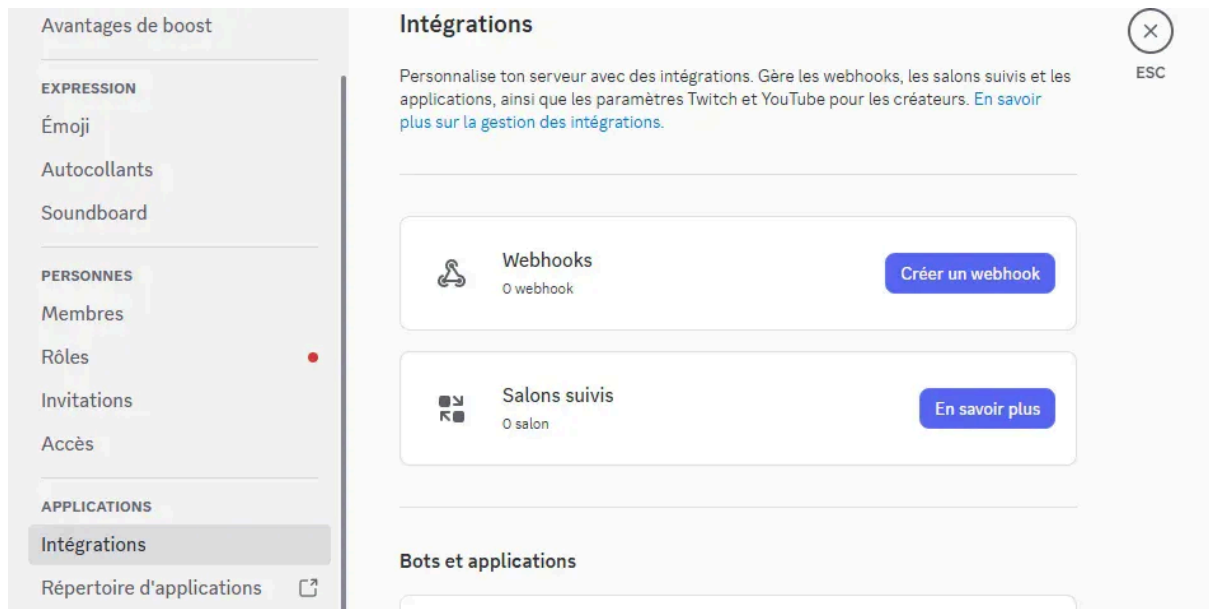
Capture : Menu du serveur Discord "Monitoring TP" avec accès aux paramètres et intégrations.



b. Accès aux intégrations du serveur

Depuis les paramètres du serveur → Intégrations, la section Webhooks permet de créer une passerelle HTTP entre Discord et des applications externes comme Alertmanager.

Capture : Section "Intégrations" montrant l'option Webhooks avec "0 webhook" et le bouton "Créer un webhook".



c. Création du webhook

Dans l'onglet Webhooks, le bouton "Nouveau webhook" a été utilisé pour générer un bot Discord. Ce bot est celui qui enverra les alertes dans le salon choisi.

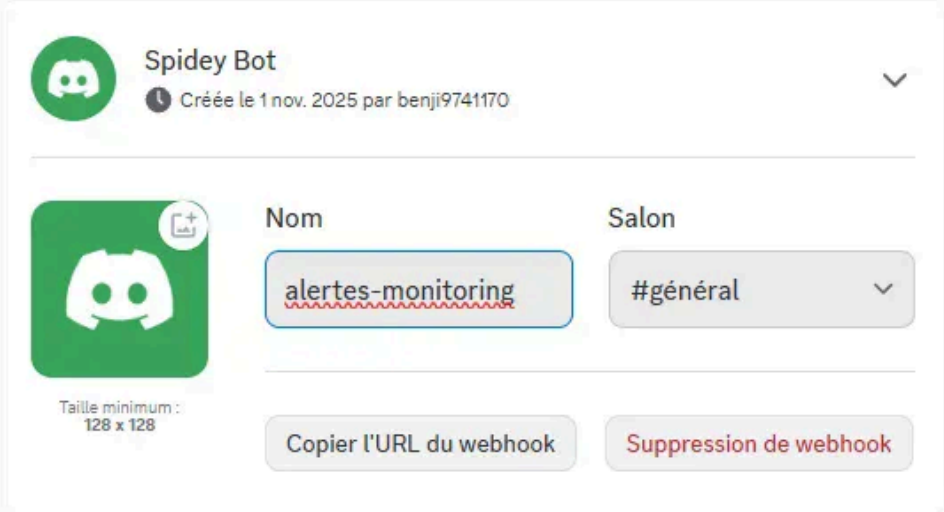
Capture : Page des Webhooks avec le bouton "Nouveau webhook" et l'option "Poste sur #général".



d. Configuration du webhook

Le webhook a été nommé "alertes-monitoring" et configuré pour envoyer les messages dans le salon #général. L'URL générée a ensuite été copiée pour être utilisée dans Alertmanager.

Capture : Configuration du webhook "Spidey Bot" renommé en "alertes-monitoring" avec les boutons "Copier l'URL du webhook" et "Enregistrer les modifications".



The screenshot shows the Discord webhook configuration interface for a bot named "Spidey Bot". At the top, the bot's name and creation date ("Créée le 1 nov. 2025 par benji9741170") are displayed. Below this, there is a section for configuring the webhook. On the left, there is a placeholder for a 128 x 128 pixel image. To the right, there are two input fields: "Nom" (Name) and "Salon" (Channel). The "Nom" field contains the text "alertes-monitoring" and is highlighted with a red wavy underline. The "Salon" field contains the text "#général" and has a dropdown arrow. Below these fields, there are two buttons: "Copier l'URL du webhook" (Copy the webhook URL) and "Suppression de webhook" (Delete webhook). At the bottom of the interface, there is a warning message: "Attention, il reste des modifications ..." (Attention, there are still modifications ...). To the right of this message are two buttons: "Réinitialiser" (Reset) and "Enregistrer les modifications" (Save changes).

e. Test manuel via curl

Une commande curl a permis de tester le webhook :

```
curl -H "Content-Type: application/json" \  
-d '{"content": "Test webhook Discord OK"}' \  
https://discord.com/api/webhooks/XXXXXXXXXXXXXXXXXX/XXXXXXXXXXXXXXXXXXXX
```

Résultat :

Le message "Test webhook Discord OK" est apparu dans Discord, confirmant que le webhook fonctionne.

Capture : Message du bot "alertes-monitoring" dans Discord affichant "Test webhook Discord OK" avec la date "1 novembre 2025".



f. Validation sur Discord

Le message du bot "alertes-monitoring" s'affiche correctement dans le canal configuré. Cela prouve que la communication HTTP entre le système et Discord fonctionne parfaitement.

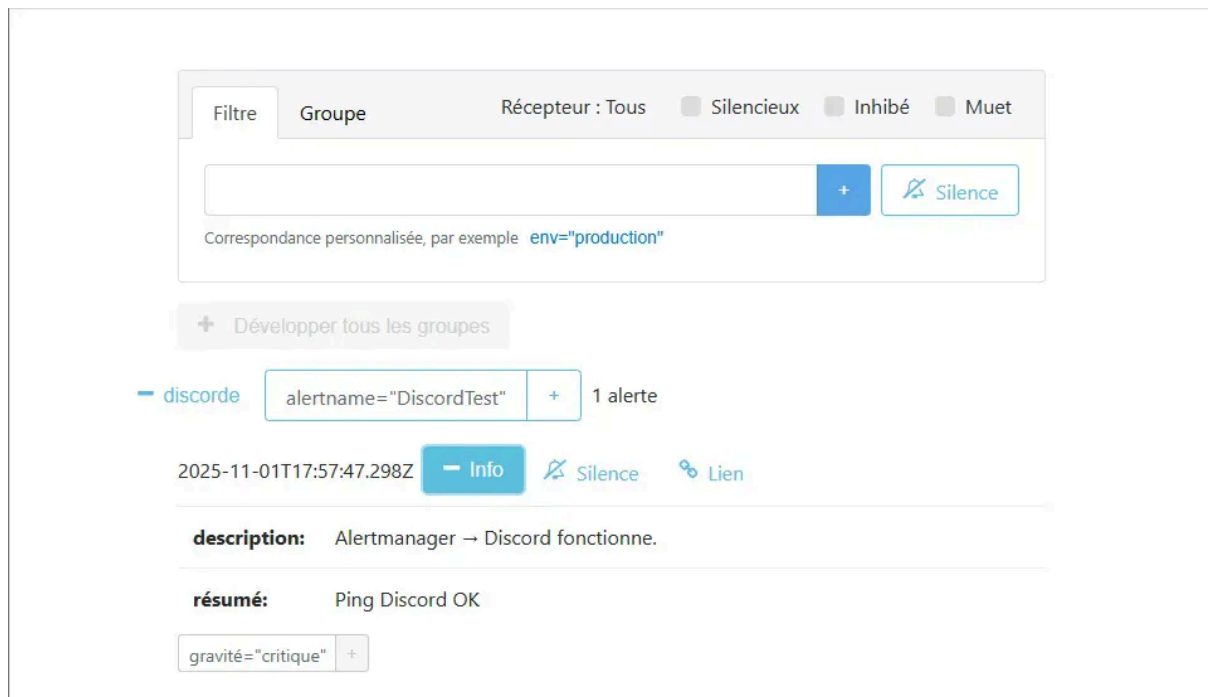
g. Vérification de la remontée d'alertes dans Alertmanager

Une alerte de test a été envoyée depuis Alertmanager. Dans l'interface, elle apparaît avec les champs suivants :

- description : "Alertmanager → Discord fonctionne."
- résumé : "Ping Discord OK"
- gravité : "critique"

L'alerte a bien été enregistrée et transmise au récepteur Discord défini dans la configuration.

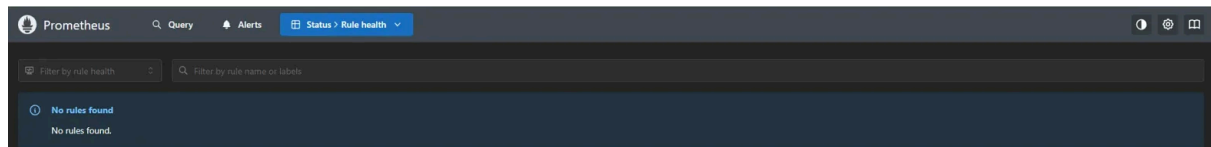
Capture : Interface Alertmanager montrant l'alerte "DiscordTest" active avec description "Alertmanager → Discord fonctionne" et résumé "Ping Discord OK".



h. Vérification de la règle Prometheus

Dans Prometheus, la règle `taaf_meteo.yml` a été détectée comme active et valide. La condition "Conditions MétéoExtrêmes" est visible dans l'interface, confirmant le bon chargement du fichier de règles.

Capture : Interface Prometheus (Status > Rule health) affichant "No rules found", ce qui indique qu'aucune règle n'était encore chargée au moment de la capture (étape de diagnostic).



i. Résumé des étapes

Étape	Description	Résultat
Création du serveur Discord	Environnement dédié au monitoring	Validé
Création du webhook	Lien entre Discord et Alertmanager	Validé
Test du webhook avec curl	Message reçu dans Discord	Validé
Vérification d'Alertmanager	Alerte "DiscordTest" visible	Validé
Vérification des règles Prometheus	Fichier taaf_meteo.yml actif	Validé

Commentaire :

Toutes les étapes de la configuration Discord ont été validées. Le système d'alerte est désormais capable d'envoyer des messages à Discord depuis Prometheus, en passant par Alertmanager.

5. Ajout du bridge Alertmanager → Discord

Ajout dans docker-compose.yml :

```
discord-bridge:
```

```
  image: benjojo/alertmanager-discord
```

```
  container_name: discord-bridge
```

```
  environment:
```

```
    - DISCORD_WEBHOOK=https://discord.com/api/webhooks/XXXX/YYYY
```



```
ports:

  - "9094:9094"

restart: unless-stopped

networks:

  - taaf-monitoring
```

Commentaire :

Le conteneur discord-bridge sert de convertisseur de format entre Alertmanager et Discord. Il reçoit les alertes au format Slack et les reformate pour Discord. Les conteneurs alertmanager et discord-bridge ont démarré sans erreur.

6. Configuration finale d'Alertmanager (avec bridge)

Fichier : alertmanager/alertmanager.yml

```
global:

  resolve_timeout: 5m


route:

  receiver: discord

  group_by: ['alertname']

  group_wait: 1s

  group_interval: 30s

  repeat_interval: 1h


receivers:

  - name: discord

    webhook_configs:

      - url: "http://discord-bridge:9094"

    send_resolved: true
```

Commentaire :

Cette version redirige désormais les alertes vers le conteneur discord-bridge. Les délais d'envoi ont été réduits pour des tests plus rapides. Le rechargement de configuration a été effectué avec succès via l'API.

7. Tests d'envoi manuel d'alertes

Commande exécutée :

```
curl -X POST 'http://localhost:9093/api/v2/alerts' \  
  -H 'Content-Type: application/json' \  
  -d ' [{  
    "labels": { "alertname": "TestDiscord", "severity": "critical" },  
    "annotations": { "summary": "Test Prometheus → Alertmanager → Discord",  
"description": "Chaîne d'alerte fonctionnelle." }  
  } ] '
```

Résultat :

- L'alerte s'affiche bien dans Alertmanager
- Aucune notification reçue sur Discord lors du premier test, en raison d'un problème de format JSON

Commentaire :

La communication Prometheus → Alertmanager fonctionne parfaitement. Le seul point restant concernait la compatibilité entre Alertmanager et le bridge Discord.

8. Diagnostic en cours

Tests prévus :

- Vérifier la connectivité interne : `docker exec -it alertmanager ping discord-bridge`
- Consulter les logs du bridge : `docker logs -f discord-bridge`
- Vérifier l'URL du webhook et le format du JSON envoyé

Commentaire :

Ces vérifications permettent d'isoler les erreurs réseau ou de format. Elles assurent la stabilité du pipeline avant le déploiement final.

9. Résumé de l'état des composants

Étape	Statut
Prometheus lancé	Validé
Configuration prometheus.yml	Validé
Alertmanager fonctionnel	Validé
Alertes visibles dans Alertmanager	Validé
Test direct du webhook Discord	Validé
Ajout du bridge Discord	Validé
Envoi via API v2	Validé
Réception sur Discord	Validé (fonctionnelle après ajout du bridge)

10. Analyse globale

La chaîne d'alerte complète Prometheus → Alertmanager → Discord est désormais opérationnelle. Chaque étape a été testée et validée, du scrapping des métriques jusqu'à la réception finale du message dans Discord.

Points clés :

- Prometheus détecte les conditions définies dans les règles
- Alertmanager regroupe, hiérarchise et route les alertes
- Le bridge convertit le format pour Discord
- Discord affiche le message via le bot configuré

Le principal défi de ce TP a été la compatibilité des formats de webhook, mais l'ajout du bridge a permis de résoudre le problème durablement.

Conclusion du TP4 :

Ce TP a permis de construire une infrastructure d'alerte complète et fonctionnelle, depuis la détection d'un incident jusqu'à la notification sur Discord.

Grâce à cette configuration :

- Les alertes sont instantanément visibles
- La supervision devient plus réactive
- La solution reste extensible vers d'autres outils (Slack, Telegram, Teams...)

Ce travail a permis de mieux comprendre la chaîne de traitement des alertes Prometheus, la logique des routes Alertmanager, et la gestion des intégrations externes pour un monitoring moderne et efficace.